

ELEC 475 Lab 4, Alistair Barfoot and Luke Barry

Contents

1	Introduction	2
2	Methodology	2
2.1	Model Architecture	2
2.2	Dataloading	3
2.3	Training Loop Design	4
3	Results	5
3.1	Base Model	5
3.2	Augmentation Model	7
3.3	Regularization Model	8
3.4	Combined Model	9
3.5	Natural Language Processing	10
3.6	Zero-Shot Classification	11
4	Discussion	12
4.1	CLIP Loss	12
4.2	Model Performance	13
4.3	Future Improvements	13
4.4	LLM Usage	14
5	Conclusion	15

1 Introduction

CLIP [3] is a framework which essentially connects images to text captions which describe key details of the corresponding image. Conceptually, every image and caption are in a vector space and then learning puts the matching pairs close together in the space. For a batch of N images and N captions, the model produces an $N \times N$ similarity matrix by comparing every image embedding with every caption embedding. The CLIP model produces an embedding for each image and caption sample, then compares the images to the captions. Any non-diagonal entry represents an image paired with an incorrect caption. The goal for training is to shape the matrix so that the matching image-caption pairs look similar and the non-matching pairs do not look similar.

CLIP learns through matching the pairs together and pushing the non-matching pairs apart. This essentially shapes the similarity matrix so that the correct pairs have high similarity and the incorrect pairs have low similarity.

Because CLIP learns from images and natural language it can perform a range of tasks from text based image retrieval, image-text reasoning and zero-shot classification, where the model can classify new images into categories it was not trained with. CLIP is trained on 400 million images which is far larger than that of other datasets like ImageNet [2]. CLIP is a highly adaptable model for a plethora of computer vision tasks including object recognition, OCR, and can be used directly with natural language queries.

2 Methodology

There are three main processes within the fine-tuning of the model for the CLIP architecture: the model architecture, the dataloading and the training loop which all work hand-in-hand.

2.1 Model Architecture

There is both an image and text encoder which work together for the model. The image encoder uses a *ResNet50* [2] backbone with pre-trained *IMAGENET1K_V2* weights. The model is projected to a fully connected hidden layer with 2048 nodes, followed by a GELU activation function. It is then projected to a fully connected layer mapping to 512 nodes, the dimension of the CLIP embedding space. Between the hidden layer and the activation function there is optional functionality for batch normalization and dropout between the activation function and final layer. There are 28,753,473 trainable parameters for the image encoder, which is the target of the fine tuning in this project

The text encoder is a pretrained CLIP [3] model from OpenAI's *ViT-B/32* model which maps to an embedding dimension of 512, similar to the image encoder. The weights are frozen and not updated through the training of the model. There are 88,224,232 frozen parameters within

the text encoder.

There is a learnable temperature parameter at the logit layer between the image and text encoder.

The cosine similarity matrix is calculated by multiplying the image embedding vector by the transpose of the text embedding vector, which is multiplied by the temperature parameter. The image-to-text loss is calculated by the cross entropy loss of the similarity matrix and the labels, while the text-to-image loss is calculated by the cross entropy loss of the matrix transformed with the labels. The average of these two losses is taken as the InfoNCE loss.

2.2 Dataloading

The COCO dataset is downloaded onto the machine from an external source, with the root of the folder called **archive**. A text file contains the global path to the archive folder which tells the dataloader where to point to. The images are found in the directory:

```
archive/coco2014/images/{train/val}2014.
```

For this report, the test files were not used as they didn't come with labels.

Since the images are in separate folders, the dataloader reads all the images in the directory. A JSON file is found at the path:

```
archive/coco2014/annotations/instances_{train/val}2014.json
```

This provides the image filename to point to, with the bounding box locations and categories. If images have no annotations, they are ignored which reduces the size of the dataset.

One of each category is kept as labels, so if the available labels are “person”, “vehicle”, “dog”, “person” for example only [“person”, “vehicle”, “dog”] are retained. Since captions are not provided in the dataset, the caption is made by concatenating one of each label with a space in between.

The images are loaded using the pillow library. Then, the transforms are applied. In the case where augmentations are not used, they are resized to (224, 224), converted to a tensor, and normalized using standard CLIP mean and std. An option for augmentation is available, which transforms using color jitter and gaussian blur between the resizing and conversion to tensor. The batch is then synthesized by putting N (batch size) images in a torch stack. The images in the training set are shuffled for different training ordering at each epoch. The dataloader creates a validation set using the similar procedure, but without color jitter or gaussian blur transforms and without shuffling.

2.3 Training Loop Design

The training loop parses arguments for adjusting the learning rate, number of epochs, whether to use augmentation/regularization, whether to use text caching, and functionality to resume training a model. If text caching is used, the program looks for an existing *.pt file containing the cached text embeddings. If this doesn't exist, the program caches all of the text and writes it to a file.

The InfoNCE loss is calculated with the procedure outlined above, and Recall@K for $K=[1,5,10]$ is calculated on each image in the validation set to monitor model effectiveness and training progress.

An AdamW optimizer is used which updates the parameters after each batch with an adjustable weight decay which can update the learning rate if necessary. Then the weights are updated and each validation image is sent through the model and the loss is calculated. If the validation loss reaches a global minimum, then the .pth file is saved and patience reset to zero and if not then the patience counter is incremented by one. When the patience counter reaches 5 the program exits out of the loop. After each epoch, the cuda cache is cleared.

3 Results

Four models were trained on the architecture each with modifications to the training. Modifications between the models included batch normalization which normalizes the inputs to a layer, allowing the training to be faster and the gradient descent to converge faster, Dropout which deactivates nodes within the model during each training iteration to create smaller networks inside the model that each need to learn the task, and augmentation which essentially expands the dataset by creating modified images of ones that already exist.

Hyperparameter	Value
Learning Rate	5e-5
Epochs	10
Batch Size	16
Image Size	(224, 224)
Dropout Probability	0.1
Color Jitter	brightness=0.3, contrast=0.3, saturation=0.3, hue=0.1

Table 1: Hyperparameters used for training all models

3.1 Base Model

The base model had no augmentation or regularization. The validation InfoNCE loss overall was 0.2532 with an average inference time of 3.8ms. When running the test, multiple changes were made to try to stop the model from overfitting by changing the learning rate and weight decay, these were the best results we got.

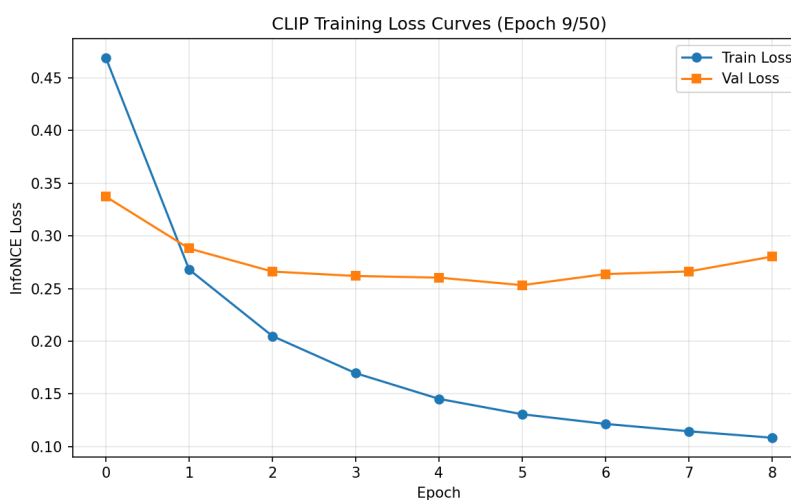


Figure 1: Validation InfoNCE Loss for Base Model

The Image-to-Text retrieval results are shown below, where each metric represents the percentage of images whose correct text description appears in the top K retrieved texts.

Model	Recall@1	Recall@5	Recall@10
Base	2.29%	7.42%	11.59%

Table 2: Image-to-Text Retrieval Results for Base Model

The Text-to-Image retrieval task, where the model retrieves images given a text query, yielded comparable performance:

Model	Recall@1	Recall@5	Recall@10
Base	2.25%	7.51%	11.96%

Table 3: Text-to-Image Retrieval Results for Base Model

To visualize the embedding space quality, Figure 2 presents the similarity matrix showing cosine similarities between all image-text pairs in the validation set. The diagonal represents correct pairings, and the off-diagonal noise indicates false matches.

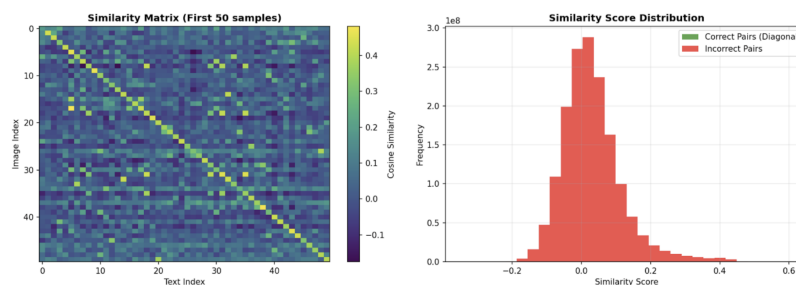


Figure 2: Similarity Matrix for Base Model Embeddings

3.2 Augmentation Model

Next augmentation was added to the model to artificially expand the dataset while creating noisier images so that the model has more to train on. However, this seemed to decrease performance as the best InfoNCE validation loss was 0.2738 with an inference time of 3.7ms.

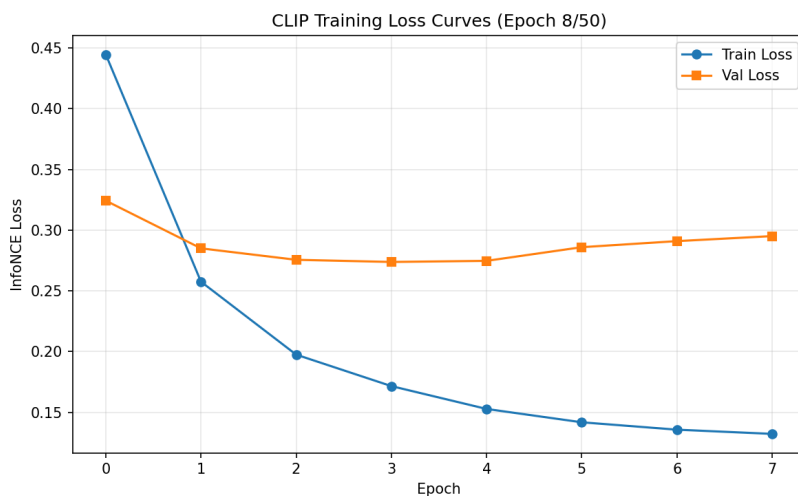


Figure 3: Validation InfoNCE Loss for Augmentation Model

The Image-to-Text results were similar to that of the model without augmentation. Performing slightly worse:

Model	Recall@1	Recall@5	Recall@10
Augmentation	1.9%	6.3%	10.0%

Table 4: Image-to-Text Retrieval Results for Augmentation Model

Similarly the Text-to-image results performed worse than the base model:

Model	Recall@1	Recall@5	Recall@10
Augmentation	2.0%	6.8%	10.7%

Table 5: Text-to-Image Retrieval Results for Augmentation Model

To visualize the similarity matrix here is the first 50 samples along with a similarity score distribution.

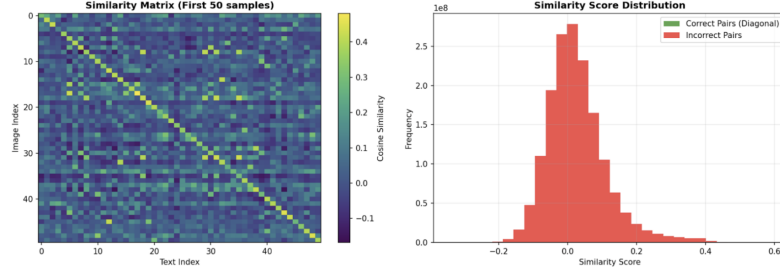


Figure 4: Similarity Matrix for Augmentation Model Embeddings

3.3 Regularization Model

This model implements regularization which includes batch normalization and dropout. The validation InfoNCE loss overall was 0.2307 with an inference time of 4.1ms.

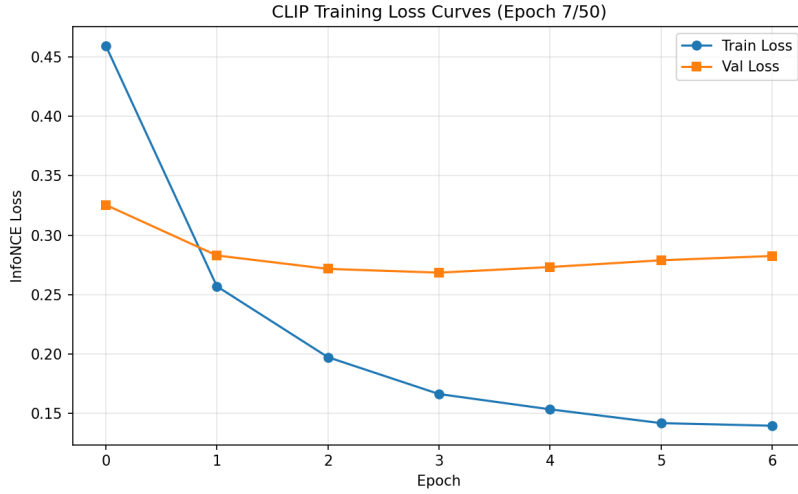


Figure 5: Validation InfoNCE Loss for Regularization Model

The Image-to-Text retrieval results are shown below, where each metric represents the percentage of images whose correct text description appears in the top K retrieved texts.

Model	Recall@1	Recall@5	Recall@10
Regularization	2.63%	8.42%	12.96%

Table 6: Image-to-Text Retrieval Results for Regularization Model

The Text-to-Image retrieval task, where the model retrieves images given a text query, yielded comparable performance. Overall, doing better than the two models before.

Model	Recall@1	Recall@5	Recall@10
Regularization	2.72%	8.62%	13.50%

Table 7: Text-to-Image Retrieval Results for Regularization Model

To visualize the embedding space quality, Figure 6 presents the similarity matrix showing cosine similarities between all image-text pairs in the validation set. The diagonal represents correct pairings, and the off-diagonal noise indicates false matches.

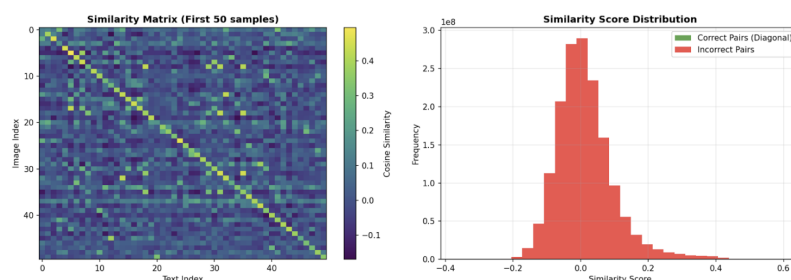


Figure 6: Similarity Matrix for Regularization Model Embeddings

3.4 Combined Model

This model implements regularization and augmentation. The validation InfoNCE loss overall was 0.2686 with an inference time of 4.2ms.

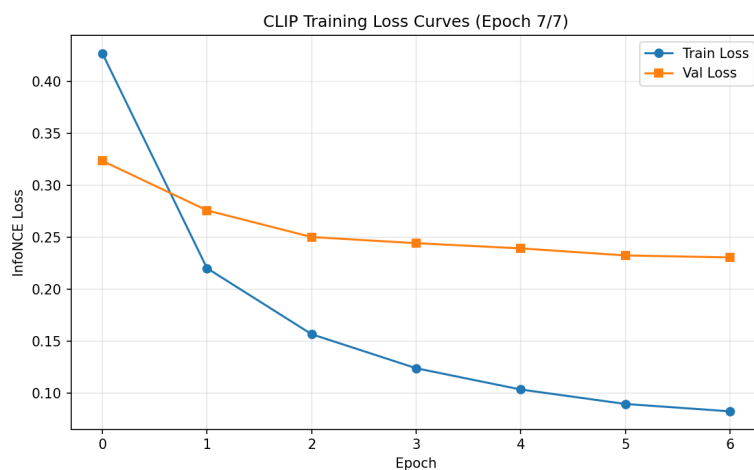


Figure 7: Validation InfoNCE Loss for Combined Model

The Image-to-Text retrieval results are shown below, where each metric represents the percentage of images whose correct text description appears in the top K retrieved texts.

Model	Recall@1	Recall@5	Recall@10
Combined	2.27%	7.51%	11.64%

Table 8: Image-to-Text Retrieval Results for Combined Model

The Text-to-Image retrieval task, where the model retrieves images given a text query, yielded comparable performance. Overall, doing better than the two models before.

Model	Recall@1	Recall@5	Recall@10
Combined	2.37%	7.69%	12.17%

Table 9: Text-to-Image Retrieval Results for Combined Model

To visualize the embedding space quality, Figure 8 presents the similarity matrix showing cosine similarities between all image-text pairs in the validation set. The diagonal represents correct pairings, and the off-diagonal noise indicates false matches.

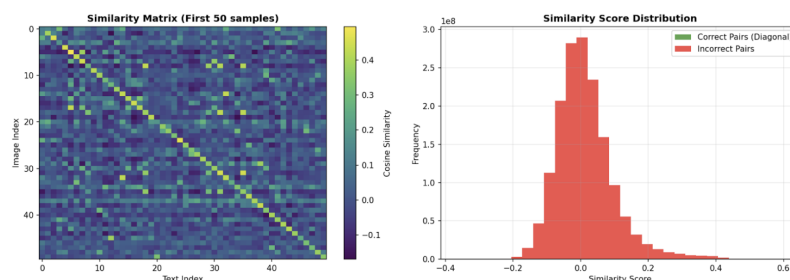


Figure 8: Similarity Matrix for Combined Model Embeddings

3.5 Natural Language Processing

To show the results of the model interpreting language. A prompt “a man surfing in the ocean” was inputted to see the text-to-image result. The graph below shows the model, along with the top 3 ranking of the matching images.

Model	Rank 1	Rank 2	Rank 3
Base			
Augmentation			
Regularization			
Combined			

3.6 Zero-Shot Classification

To show the results of the model classifying unseen classes, an image of a baseball game was shown to each of the four models. The models were each asked to specify the probabilities for ["baseball", "basketball", "football", "hockey", "soccer", "tennis"]. While the labels "baseball glove" and "baseball bat" are provided in the COCO dataset, "baseball" is not which makes this a zero-shot classification task. The results are shown below:

The results show that all four models were able to correctly classify the unseen class with full confidence.

Class	Base	Augmentation	Regularization	Combined
Baseball	100%	100%	100%	100%
Basketball	0.0%	0.0%	0.0%	0.0%
Football	0.0%	0.0%	0.0%	0.0%
Hockey	0.0%	0.0%	0.0%	0.0%
Soccer	0.0%	0.0%	0.0%	0.0%
Tennis	0.0%	0.0%	0.0%	0.0%

Table 10: Zero-Shot Classification Results for Baseball Image

4 Discussion

4.1 CLIP Loss

The motivation behind using CLIP loss in this case to fine-tune the weights of the ResNet50 model is to be able to categorize features within an image to unseen classes. The CLIP text encoder from ViT-B/32 developed by OpenAI uses a concept called attention [4] which encodes a text prompt based on three values: Key (K), Query (Q), and Value (V). For the case of this report, each label has a corresponding query and is associated with a word embedding vector. By using this encoder, we can essentially get the context of an input text prompt regardless of whether it falls into one of the pre-defined classes.

In the case of this lab, we pre-processed the available labels from a training image into a caption which was then passed through the text encoder. By encoding the image using a separate architecture designed to classify images with high accuracy, and mapping to a vector of the same size of the CLIP encoder output, we can relate the two to perform Natural Language Processing (NLP) tasks. The idea is to project a similarity matrix by computing the dot product between the two vectors and optimizing the cosine similarity values on the diagonal.

One of the major upsides of using a method like this is that the model is able to classify images with unseen categories [1]. Since the attention model provides the context of the word rather than just its direct meaning, the output might be able to classify a “hamster” in an image even if it was just trained on “cat” and “dog”. Another upside of using an attention model for the text encoding is that it provides a more human method of image retrieval, since humans are more likely to want to understand or generate a prompt “a man surfing in the ocean” than [“man”, “ocean”, “surfboard”, “waves”].

A downside of using such a method is that the time and space complexity scales quadratically ($O(n^2)$) with respect to batch size. This arises due to the requirement for the attention mechanism [4] to compute similarities between all items in the batch or sequence, resulting in an increase of computational resources required for large datasets.

4.2 Model Performance

For random selection, the expected Recall@K value is equal to K/N , where N is the number of images in the validation set. In this case, since there were 40,137 images the expected Recall@1 is around $2.5e-5$. The models outperformed this metric significantly, often by about 3 factors of 10. This suggests that the model was learning from the data provided and making reasonable answers when provided a text or image query.

Table 11 aggregates the results from the recall metrics. For comparison, the expected value for each recall metric is also provided.

Model	Image-to-Text			Text-to-Image		
	R@1	R@5	R@10	R@1	R@5	R@10
Base	2.29%	7.42%	11.59%	2.25%	7.51%	11.96%
Augmentation	1.9%	6.3%	10.0%	2.0%	6.8%	10.7%
Regularization	2.63%	8.42%	12.96%	2.72%	8.62%	13.50%
Combined	2.27%	7.51%	11.64%	2.37%	7.69%	12.17%
Expected	0.0025%	0.0125%	0.025%	0.0025%	0.0125%	0.025%

Table 11: Summary of Retrieval Results Across Models

In each of the six recall metrics, the model trained with regularization outperformed each of the other three models. Table 12 shows the relative improvement from the mean of the three models with respect to the performance of the regularized model.

Metric	R@1	R@5	R@10
Image-to-Text	22.1%	19.0%	17.0%
Text-to-Image	23.3%	17.5%	16.3%

Table 12: Relative Improvement of Regularization Model Over Mean of Other Models

It is shown that the regularization model outperforms the other models similarly between image-to-text and text-to-image. Where the regularization model especially excels, however, is in the Recall@1 metric, where it was almost a quarter more effective at classifying labels or images.

4.3 Future Improvements

In a paper published in 2012 by the creators of the *AlexNet* [2] convolutional neural network, the authors suggested that “All of our experiments suggest that our results can be improved simply by waiting for faster GPUs and bigger datasets to become available”

In the case of our model architecture, it is apparent that is not the case. Since 2012, the state of GPU production has enhanced significantly as training computer vision models has

become more accessible and prevalent in industry. Since the model started overfitting after the first epoch for each training run, this suggests that architecture and pre-processing changes need to be made for improvement of the Recall@K metrics. The *ResNet50* model used for the encoding of the images is already an outdated architecture, designed in 2015 to be used on older hardware. By using a deeper model, it may be possible to generalize more from the data.

Further, it was shown that regularizations had a positive impact on the effectiveness of the model while augmentations had a negative impact. Further ablation studies should be done which tests the effectiveness of applying random crops, flipping, and rotating. This can be tested with altering the dropout and batch normalization parameters to see which has the most positive impact on the generalization of the model.

4.4 LLM Usage

For this project, the LLM used was GitHub Copilot which is a coding assistant embedded into the VS Code IDE. GitHub Copilot actually ships with multiple available LLM agents, the most prominent ones used for this report being “GPT-5”, “Claude Sonnet 4”, and “Claude Sonnet 4.5”. For this project, it was found that the best way to communicate with the LLM was to provide specific instructions and to not give it everything at once. For example, to implement the CLIP architecture, a prompt such as “make a PyTorch model with a *ResNet50* background that fine-tunes the weights to align to a pretrained transformer in a CLIP-style architecture.”

It was important to check at every step in the process that the LLM was generating reliable and well-written code. This process consisted of both manual review and generating fallbacks and exceptions to take care of error handling. We would read the discussion generated from the LLM to ensure that it was on the right path, and if we did not like the explanation it gave we would undo its changes. We found some models to be better at different tasks, such as GPT-5 for debugging as it provides a ‘to-do list’ so we could verify it was not making unwanted changes.

When we had issues with the code and something was not working as we wanted, we gave the file or code snippet as context to the model as well as terminal error messages, along with our suggested fix to lead the LLM in the correct direction. As well, we used the LLM to provide suggestions when our model was not generalizing the data well enough, and it would suggest things such as altering the learning rate or changing the weight decay. Ultimately, every line of code that was suggested by the LLM was reviewed and tested to ensure its feasibility.

Since we used an IDE-based LLM, we are not able to provide the link to the discussion. However, we can provide a link to our GitHub repository to show the evolution of our changes.

<https://github.com/alistair-barfoot/elec475-labs>

5 Conclusion

Training and evaluating a CLIP-style architecture provided valuable insight into how jointly learned imagetext representations behave in practice. Compared to previous computer vision tasks, this project required managing significantly larger datasets, integrating a text encoder, and carefully aligning two different embedding spaces. When experimenting with multiple different configurations it was clear that regularization techniques proved to be positively impactful whereas data augmentation degraded the performance.

Although all models significantly did better than a random choice, *ResNet50* backbone proceeded to show clear limitations in its functionality, with each model overfitting extremely quickly. A more modern computer vision backbone could have yielded better recall performance rather than tinkering with hyperparameter adjustments within *ResNet*. More ablation studies into different augmentation and regularization techniques could have been conducted in order to find which ones positively impact the loss.

Beyond the model, we learned more into the importance of effective LLM assistance when building complex projects. We continued to improve upon our own coding while also understanding the limitations of what the LLM could do. With iterative validation and manual overview the project was done faster than how it would have if written by hand while also keeping the same level of quality.

References

- [1] Muhammad Ali and Salman Khan. “CLIP-Decoder : ZeroShot Multilabel Classification Using Multimodal CLIP Aligned Representations”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*. Oct. 2023, pp. 4675–4679.
- [2] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira et al. Vol. 25. Curran Associates, Inc., 2012. URL: https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf.
- [3] Alec Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *CoRR* abs/2103.00020 (2021). arXiv: 2103.00020. URL: <https://arxiv.org/abs/2103.00020>.
- [4] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.